

INDEX

S.No.	Topic	Page No.
1.	Title Page	2
2.	Aim, Objective & Theory	3
3.	Tools Required & Methodology	4
4.	Code	5
5.	Output	6
6.	Learning Outcomes	7

**DEPARTMENT OF
AIT-CSE
CHANDIGARH UNIVERSITY**



Practical File for Exp: 4

Subject Name: AML

Subject Code: 24CSH-338

Submitted by:

Name: KHUSHI

UID: 24BAI70800

Section: 24AML-11(B)

Submitted to:

Mr. Aaskaran Bishnoi

Assistant Professor

Date	LW (12)	VW (10)	FW(8)	TM (30)

Aim:- To implement a **Self-Organizing Map (SOM)** using Python and apply it for clustering and visualization of high-dimensional data.

Objective :-

- . To understand the concept of Self-Organizing Map.
- . To implement SOM using Python.
- . To apply SOM on high-dimensional data.
- . To cluster similar data points using SOM.
- . To visualize high-dimensional data on a two-dimensional map.
- . To analyze the role of Best Matching Unit and neighborhood learning.

Theory :-

A **Self-Organizing Map (SOM)** is an unsupervised neural network technique used for clustering and visualization of high-dimensional data. It is also known as a **Kohonen Map**.

SOM maps high-dimensional input data onto a low-dimensional grid, usually a two-dimensional grid, while preserving the topological relationship between data points. Similar data points are placed close to each other on the map.

SOM uses competitive learning. For each input vector, the neuron whose weight vector is closest to the input is selected. This neuron is called the **Best Matching Unit (BMU)**. The BMU and its neighboring neurons are updated to become more similar to the input vector.

The basic weight update rule is:

Where:

- . = updated weight
- . = old weight
- . = learning rate
- . = neighborhood function
- . = input vector

SOM is useful for clustering, visualization, dimensionality reduction, anomaly detection, and pattern recognition. MiniSom is a minimal NumPy-based Python implementation of Self-Organizing Maps.

Tools Required:-

- ✦ Python
- ✦ Jupyter Notebook / Google Colab
- ✦ NumPy
- ✦ Pandas
- ✦ Matplotlib
- ✦ Scikit-learn

Methodology:-

1. Import required Python libraries.
2. Load a high-dimensional dataset.
3. Normalize the dataset.
4. Initialize a Self-Organizing Map.
5. Train the SOM using input data.
6. Find the Best Matching Unit for each data point.
7. Assign cluster labels based on BMU position.
8. Visualize the SOM using a distance map.
9. Plot data distribution on the SOM grid.
10. Analyze clustering and visualization results.

Code :-

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import silhouette_score
from minisom import MiniSom

# Load dataset
digits = load_digits()
X = digits.data
y = digits.target

print("Samples:", X.shape[0])
print("Features:", X.shape[1])

# Normalize
X = MinMaxScaler().fit_transform(X)

# Create and train SOM
som = MiniSom(10, 10, 64, sigma=1.0, learning_rate=0.5,
random_seed=42)
som.random_weights_init(X)
som.train_random(X, 5000)

# Find BMUs
bmus = np.array([som.winner(x) for x in X])

# Cluster labels
labels = bmus[:, 0] * 10 + bmus[:, 1]

# Results
print("\nFirst 10 BMUs:")
print(bmus[:10])
print("\nFirst 10 Cluster Labels:")
print(labels[:10])
print("\nQuantization Error:",
      round(som.quantization_error(X), 4))
print("Silhouette Score:",
      round(silhouette_score(X, labels), 4))
```

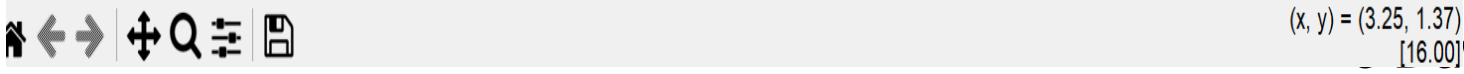
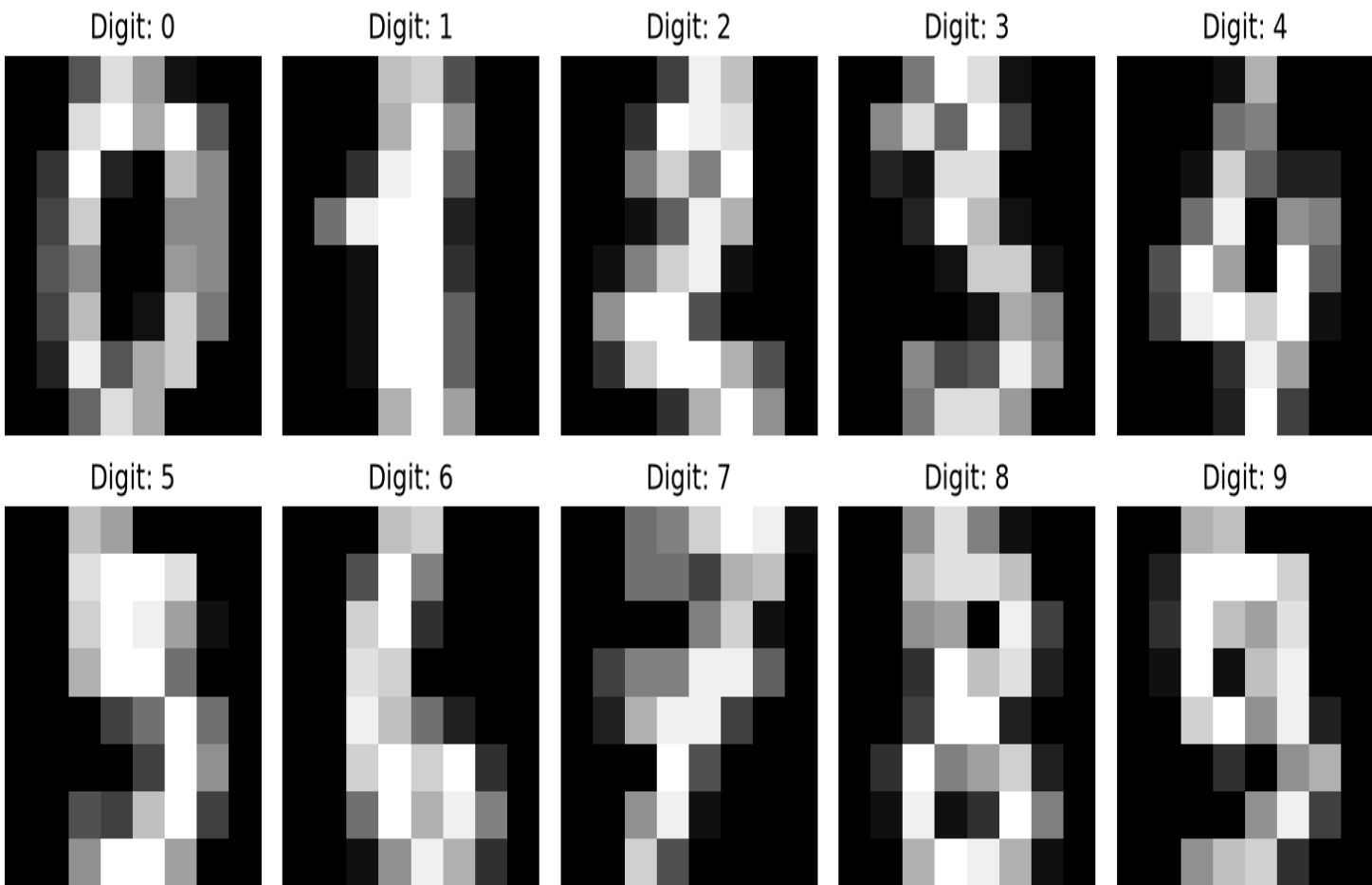
```
# SOM Distance Map
plt.figure(figsize=(8, 6))
plt.pcolor(som.distance_map().T, cmap="bone_r")
plt.colorbar(label="Distance")
plt.title("SOM Distance Map")
plt.xlabel("SOM X")
plt.ylabel("SOM Y")
plt.show()

# Map digits onto SOM
plt.figure(figsize=(8, 6))
plt.pcolor(som.distance_map().T, cmap="bone_r")
for i, (x, y_pos) in enumerate(bmus):
    plt.text(x + 0.5, y_pos + 0.5, str(y[i]),
             ha="center", va="center", fontsize=7)
plt.title("Digits Mapped on SOM")
plt.xlabel("SOM X")
plt.ylabel("SOM Y")
plt.show()
```

OUTPUT:

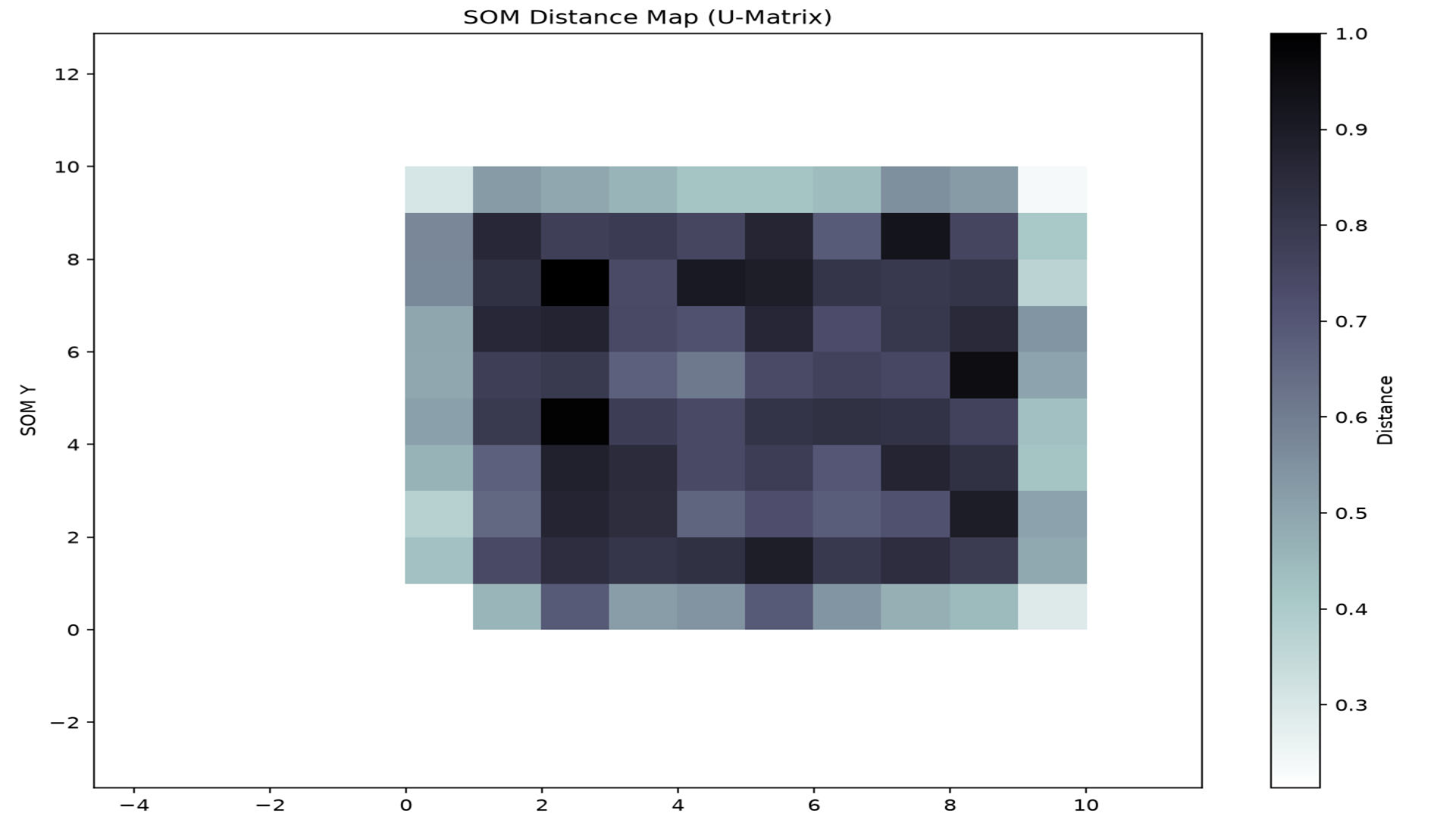
Name : KHUSHI	RESULT TABLE			
UID: 24BAI70800	-----			
=====	Actual Digit	BMU X	BMU Y	Cluster Label
SELF-ORGANIZING MAP (SOM) EXPERIMENT	0	3	6	36
=====	1	3	9	39
	2	2	7	27
DATASET INFORMATION	3	7	4	74
-----	4	1	6	16
Number of samples : 1797	5	6	1	61
Number of features: 64	6	0	0	0
Number of classes : 10	7	8	7	87
Expected:	8	5	8	58
Samples = 1797	9	6	1	61
Features = 64	0	3	6	36
Classes = 10	1	1	9	19
	2	9	2	92
DATA NORMALIZATION	3	6	7	67
-----	4	1	5	15
Minimum value after normalization: 0.0	5	3	0	30
Maximum value after normalization: 1.0	6	2	2	22
	7	9	8	98
SOM CONFIGURATION	8	5	9	59
-----	9	7	1	71
SOM grid size : 10 x 10				

Sample Images from Digits Dataset

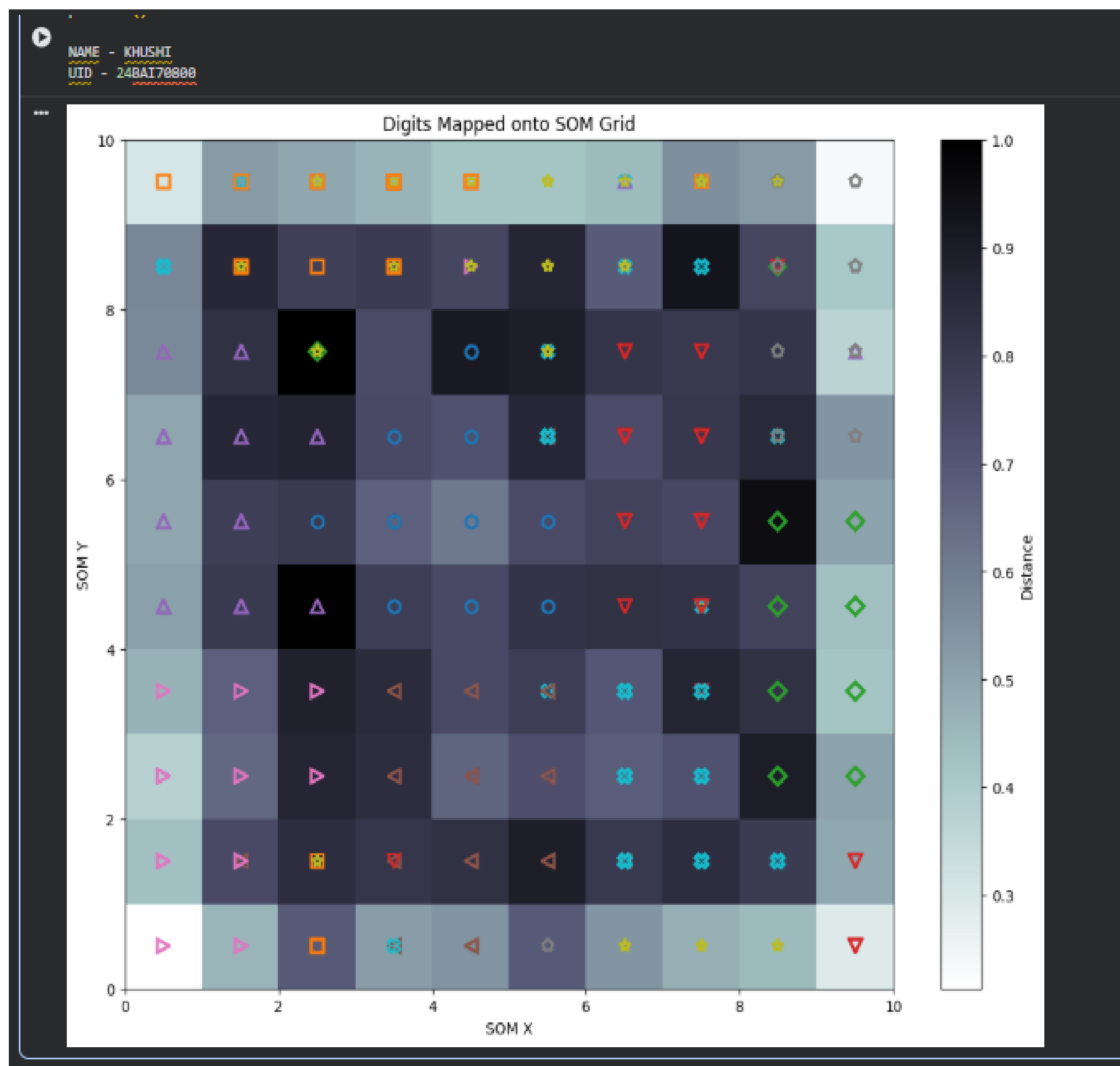


NAME - KHUSHI
UID - 24BAI70800

...	Actual Digit	BMU X	BMU Y	cluster Label	
	0	0	3	6	36
	1	1	3	9	39
	2	2	2	7	27
	3	3	7	4	74
	4	4	1	6	16
	5	5	6	1	61
	6	6	0	0	0
	7	7	8	7	87
	8	8	5	8	58
	9	9	6	1	61



Digits mapped on the SOM grid



```

from collections import Counter

bmu_counts = Counter(map(tuple, bmus))

print("Number of occupied SOM nodes:",
      len(bmu_counts))

print("\nMost populated SOM nodes:")

for node, count in bmu_counts.most_common(10):
    print(node, ":", count, "samples")

NAME - KHUSHI
UID - 24BAI70800

```

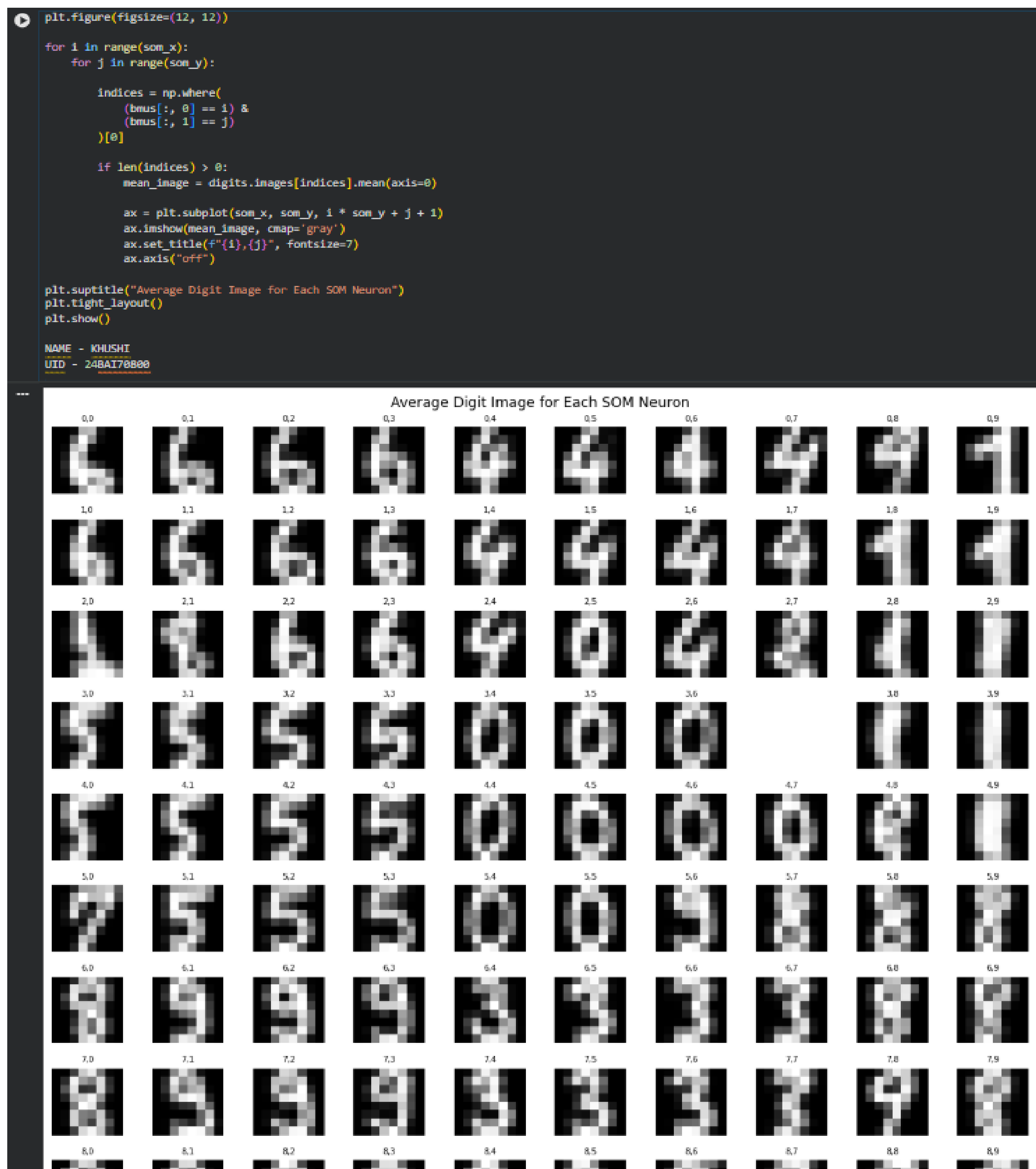
... Number of occupied SOM nodes: 99

Most populated SOM nodes:

```

(np.int64(3), np.int64(9)) : 37 samples
(np.int64(0), np.int64(2)) : 35 samples
(np.int64(1), np.int64(4)) : 30 samples
(np.int64(8), np.int64(4)) : 30 samples
(np.int64(9), np.int64(9)) : 29 samples
(np.int64(9), np.int64(4)) : 29 samples
(np.int64(6), np.int64(1)) : 28 samples
(np.int64(3), np.int64(5)) : 28 samples
(np.int64(6), np.int64(0)) : 28 samples
(np.int64(1), np.int64(5)) : 27 samples

```



Learning outcomes:

- ☐ Understood the concept and working principle of a Self-Organizing Map (SOM) as an unsupervised neural network technique.
- ☐ Learned how competitive learning identifies the Best Matching Unit (BMU) for a given input vector.
- ☐ Understood the role of neighborhood function and learning rate in updating weights of the BMU and its neighboring neurons.
- ☐ Gained hands-on experience implementing a SOM in Python using the MiniSom library.